

Software Testing Project – Reflection Report

Azra Esil Altuntaş 240717604

While working on this project, using different testing techniques helped me discover several bugs in the starter code. One of the most interesting bugs appeared during Task 4 when I used Property-Based Testing with the jqwik framework. I wrote a test for the “Cart Commutativity” property, and jqwik generated an unexpected edge case. It created two different product objects with the same ID but different prices. The shopping cart system could not handle this situation correctly, so the total price changed depending on the order in which the products were added. In Task 1 (Specification-Based Testing), testing boundary values also showed that the calculator did not properly prevent negative quantities before we added contract assertions.

Among all the techniques, I think Property-Based Testing was the most efficient considering the amount of effort required. For Structural and Specification-Based testing, I needed to write many separate test cases manually in order to cover different branches and boundary conditions. With Property-Based Testing, however, I only needed a small setup and a few simple properties such as Identity and Boundedness. After that, the framework generated many random inputs automatically. It was able to detect the ID collision bug, which is something I probably would not have considered while writing manual tests. In my opinion, it provided a large amount of coverage with relatively little code.

When I look at the testing pyramid discussed in Chapter 1, I can see that my current test suite mostly belongs to the bottom layer, which is Unit Testing. I tested classes such as PriceCalculator, ShoppingCart, and OrderProcessor independently from each other. However, the middle and top layers are still missing. I currently do not have Integration Tests or End-to-End (E2E) tests that verify whether the system works correctly with a real database or an actual user interface.

I used mocking frequently in Task 5 while testing the OrderProcessor class. Mocking was useful because it allowed me to test the business logic without needing a real payment service or inventory database. It also made it easier to simulate scenarios such as declined payments or out-of-stock products. On the other hand, mocking also has limitations because it does not test real communication between systems. For example, if the actual payment API changes its response format, the mocked tests may still pass even though the real system would fail. Because of this, mocking alone is not enough when testing interactions with external services.

If I had one more week to continue this project, I would focus on testing the integration layer. I would write Integration Tests to check how the OrderProcessor behaves with a real temporary database instead of mocks. I think this would be useful because unit tests only verify the logic of individual components in isolation. Integration testing would help confirm that the system also works correctly when real SQL queries and external connections are involved.